

High-Performance Order Matching Engine — Project Overview

C++20 Order Matching Engine | 93 headers | 14 source files | 103 test files | 524 CTest targets

A C++20 low-latency matching engine drawing on exchange design principles — **237 ns P50 core-matching latency (Clang PGO) validated on x86 Xeon bare metal** (≈ 125 ns on Apple Silicon). Thread-per-symbol partitioning is designed for horizontal scaling; **no scaling curve has been measured**, so treat that as an architectural property rather than a demonstrated one.

1. Executive Summary

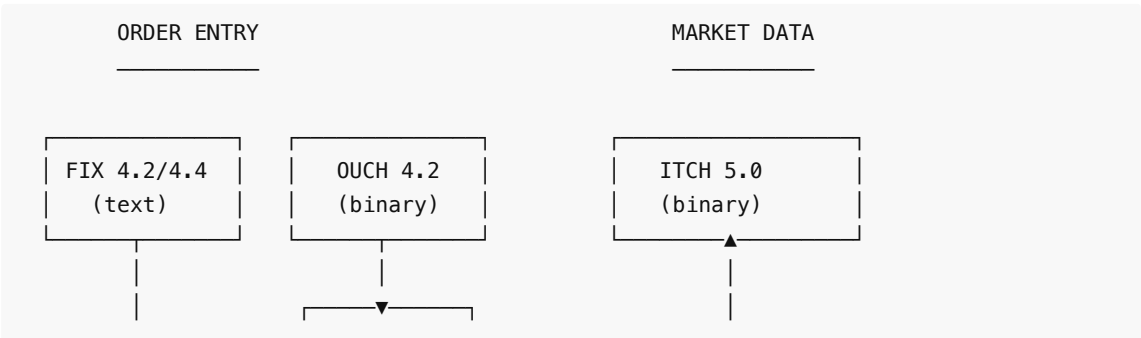
This is a C++20 low-latency order matching engine drawing on exchange design principles. It implements $O(1)$ price-level lookup via `FlatPriceMap`, lock-free MPSC queues, thread-per-symbol horizontal scaling, CRC-32 journaling with deterministic replay, four wire protocols (FIX 4.2/4.4, OUCH 4.2, ITCH 5.0, SBE) over both real TCP and UDP transports, MoldUDP64 multicast with gap-recovery retransmission service, TLA+-verified safety invariants, cross-host log replication, and an operational stack including config management, Prometheus metrics, and Docker deployment. (Webhook alerting is implemented but **cannot deliver**: `AlertDispatcher` has no TLS client, so `addWebhook()` refuses any `https://` URL — which is every endpoint worth configuring. See §11.)

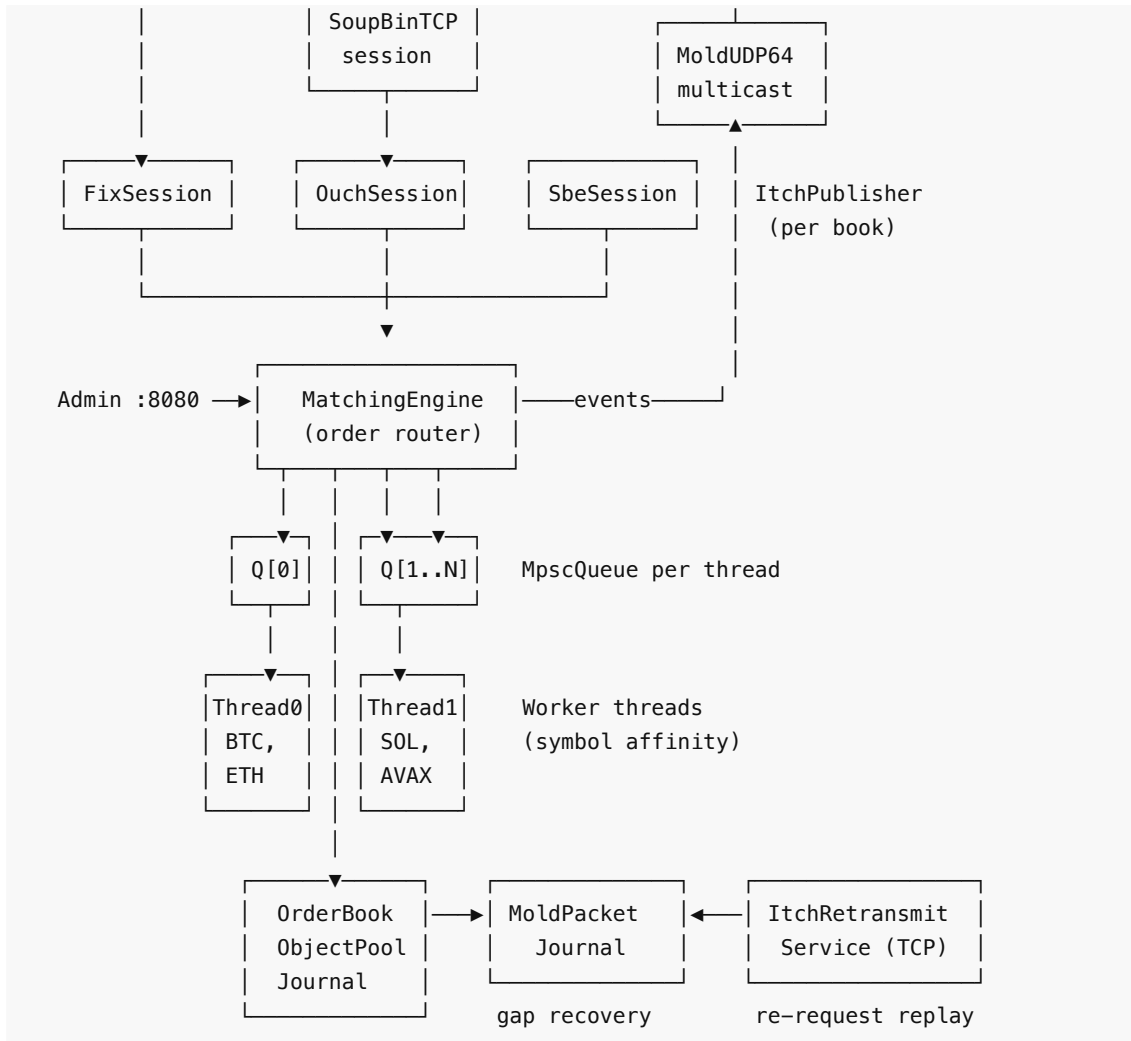
Codebase Statistics

Metric	Count
Header files (<code>include/</code>)	93
Source files (<code>src/</code>)	14
Test files	102
Individual test cases	524 CTest targets
TLA+ specifications	12 (171M distinct states — matching + replication layers)
Documentation files	8 in <code>docs/</code> (plus architecture/benchmark docs)

2. Core Engine Architecture

System Topology





Order Types Supported

- **Standard:** Limit, Market
- **Time-In-Force:** IOC (Immediate-or-Cancel), FOK (Fill-or-Kill), DAY, GTD (Good-Til-Date)
- **Conditional:** Stop, StopLimit, TrailingStop
- **Institutional:** Pegged, Iceberg (hidden quantity), Hidden, PostOnly, MIT, MOC, LOC
- **Match Algorithms:** Price-Time FIFO and Pro-Rata allocation with LMM/DMM floor guarantee (40% of available qty) and rounding-remainder priority

Core Data Structures

Component	Purpose	Complexity
FlatPriceMap	Price-level lookup by tick index	O(1) insert/lookup
FlatHashMap	Robin-Hood open-addressing hash map	O(1) amortized
IntrusiveList	Doubly-linked order queue per price level	O(1) insert/remove
ObjectPool	Pre-allocated slab allocator for Order nodes	O(1) alloc/dealloc
RingBuffer	Cache-line aligned lock-free ring buffer	O(1) push/pop

MpscQueue	Multi-producer, single-consumer lock-free queue	O(1) push/pop
-----------	---	---------------

3. Concurrency & Networking

Thread-Per-Symbol Partitioning

N worker threads with independent lock-free `MpscQueue` ring buffers. Orders are deterministically routed by `hash(symbolId) % numThreads`, so **the matching path** carries no cross-thread contention: one symbol is owned by one worker and two workers never touch the same book.

Two qualifications, because the sentence above used to read "eliminating cross-thread contention on the hot path" full stop, and that is not true of the engine as a whole:

- **Durability reserialises everything.** Every worker appends to one journal behind one `journalMutex_`, with the `fdatsync` inside the critical section. With journaling on, N workers funnel through a single lock and a single `fsync` stream — measured at roughly 165–190k appends/s on x86 CI regardless of worker count (§11). Contention-free matching and a serialised durability path are both true at once.
- **Hash routing distributes symbols uniformly, not load.** Real order flow is Zipf: one instrument can carry orders of magnitude more traffic than the long tail, and `hash(symbolId) % numThreads` will happily put it on a worker that then saturates while others idle. There is no migration path for a hot symbol short of a drain. The scaling curve in §8 is measured with **uniform** symbol load, which is the shape that hides this, and it is not a claim about venue-shaped flow.

Multi-Protocol Order Entry

All four protocols dispatch into the same `MatchingEngine`.

- **FIX 4.2 / 4.4:** ASCII text + checksum, full session-layer state machine. `TransactTime` validation and version negotiation per accepted `BeginString`.
- **OUCH 4.2:** NASDAQ binary order entry, big-endian fixed-width.
- **SBE (Simple Binary Encoding):** FIX-TG schema-driven binary (CME / ICE style). Forward and backward compatibility. Encode speed of ~1 ns/op.
- **SoupBinTCP:** TCP envelope wrapping OUCH on the wire.

Market Data Stack

- **ITCH 5.0 Publisher:** Converts engine events to ITCH 5.0 frames.
- **MoldUDP64 Multicast:** Batched publisher with MTU auto-flush + gap-detecting subscriber.
- **Gap Recovery:** Bounded ring of journaled MoldUDP64 messages backing a SoupBinTCP-over-TCP retransmission service.
- **Shared Memory IPC:** L2 market data distribution to co-located consumers via `shm_open`.

Kernel-Bypass Ingestion (DPDK / F-Stack)

Optional kernel-bypass order-entry path behind `#ifdef OB_HAVE_DPDK` (CMake `-DENABLE_DPDK=ON`): `DpdkGateway` runs F-Stack (DPDK userspace TCP over the AWS ENA PMD) on a dedicated **secondary ENI**, leaving the primary interface kernel-managed so SSH is never lost. A **single RX queue / single busy-poll lcore** preserves FIFO submit order (RSS splitting one connection across lcores would reorder and break price-time priority). F-Stack owns the NIC and runs FreeBSD's TCP stack internally; the gateway consumes its BSD-socket API (`ff_recv`), so ordered TCP payload flows straight into `OuchSession::feed()` and the engine submit path **unchanged** — DPDK replaces only how bytes arrive from the NIC, not how they are parsed. When `OB_HAVE_DPDK` is undefined (the default, and every non-Linux build) the kernel

`TcpGateway` is the only ingestion path and the binary is byte-for-byte identical. **Status: written, never executed.** No part of this path has run. The "local verification" is only that macOS builds cleanly with the DPDK path `#ifdef 'd` out and the kernel-gateway tests stay green — that exercises the *absence* of the path, not the path itself. There is no measurement, and no evidence that kernel bypass improves latency on this system; treat it as unvalidated code until `scripts/dpdk_aws.sh` produces a DPDK-vs-kernel-TCP comparison on identical hardware and workload (**Session 2**). The kernel-TCP wire-to-wire baseline below is measured and stands on its own; nothing in it depends on this path.

Wire-to-Wire Latency Measurement

A two-host OUCH-over-TCP harness — `tools/WireLatencySender` (instance A) and `tools/WireLatencyReceiver` (instance B) — measures NIC-to-NIC order-entry latency. On Linux it captures NIC **hardware** timestamps via `SO_TIMESTAMPING` (TX from the socket error queue, RX from the `recvmmsg` `cmsg`), with a `CLOCK_MONOTONIC` **software fallback** on non-Linux / non-HW hosts (each CSV row records which source it used). One-way latency is estimated as **round-trip / 2**, which sidesteps clock synchronisation entirely — both timestamps are on the sender's own clock, so no PTP is needed for a first-pass number. The receiver also supports the F-Stack path (`--conf`) so the DPDK and kernel sessions emit directly comparable CSV, and a `--software-timestamps` flag forces both onto an identical `CLOCK_MONOTONIC` basis. **Implemented and baseline-validated on AWS:** a two-instance run (two `c6in.metal` , same AZ, kernel TCP, 10,000 OUCH orders, software timestamps on `CLOCK_MONOTONIC`) measured **P50 RTT 48.2 μ s / P99 RTT 55.3 μ s**, i.e. a **P50 one-way estimate of 24.1 μ s** (round-trip / 2). The DPDK-vs-kernel-TCP comparison (Session 2) is the next step, via `scripts/wire_latency_aws.sh` .

4. Regulatory & Risk Management

Feature	Implementation
Self-Match Prevention (SMP)	Cancel-Taker strategy — prevents wash trading
Circuit Breakers	Configurable % price-band breach enters a short LULD-style volatility auction (orders accumulate without continuous matching until the reopening uncross) rather than a hard halt; emits a <code>breaker_trip</code> audit event
OTR Monitoring	Real-time Order-to-Trade ratio tracking per participant
Kill Switch	Instant cancellation of all orders per participant across all symbols
Pre-Trade Risk Limits	Max order size, notional value, and position limits per participant
Rate Limiting	Token-bucket throttling at ingress (configurable rate + burst); <code>RateLimiter::reconfigure()</code> live-updates default rate/burst and clears all participant buckets — called on <code>SIGHUP</code>
Queue Backpressure	Rejects orders when queue exceeds configurable threshold (default 80%)
LMM/DMM Privileges	<code>ParticipantRole</code> enum; ProRata 40% floor guarantee to LMM/DMM orders; remainder priority

Order-Entry Authentication & Authorisation

Every gateway previously took the participant identity straight off the wire and trusted it, so anyone who could reach the port could trade as any participant, evade the per-participant rate limiter by rotating ids, and trip another participant's kill switch.

Layer	Implementation
Credential store	ParticipantAuth — constant-time secret comparison, one credential may cover several participant ids (a firm running multiple desks). Loaded from a dedicated file, not engine.conf, so secrets do not travel with a config that gets pasted into tickets. A malformed line fails the whole load rather than silently dropping a firm's credential
Per-protocol logon	FIX at Logon (tags 553/554), OUCH at SoupBinTCP Login, and the binary gateway via a login frame carried in the V2 header's previously-reserved flags word — so existing clients, which always sent zero, are unaffected and no version bump was needed. Unknown flag bits are rejected rather than ignored
Required by default	GatewayServer refuses to start unless credentials are configured or --no-participant-auth / OB_NO_PARTICIPANT_AUTH=1 is passed deliberately, matching the bar AdminServer already held. The check runs before the engine, publisher or listen socket exist
Authorisation	A session may only act as a participant its credential covers — including KillSwitch, which acts directly on the participant id in the request
Order ownership	Cancel/Modify/CancelReplace are checked against the resting order's owner inside the book lock. The denial is reported to the client as OrderNotFound, deliberately indistinguishable from a missing order: a distinct code would turn cancel into an order-id oracle

Enforcement follows verification — with authentication disabled the claimed participant id is an unchecked assertion, so ownership is not enforced against it either.

5. Microstructure Research Infrastructure

A self-contained quantitative research layer that runs against the live matching engine, enabling empirical microstructure analysis and strategy development.

Component	Purpose
SimulationDriver	Multi-agent market simulator (NoiseTrader, MarketMaker, InformedTrader)
MicrostructureMetrics / ResearchHarness	Snapshot collection and research session management
VPINCalculator	Volume-Synchronized Probability of Informed Trading
SpreadDecomposition	Huang-Stoll and Glosten-Harris decomposition (adverse selection, inventory, order processing components)
OrderFlowAnalytics	Roll spread, queue imbalance signal, logistic fill-probability model
ImpactModel	Almgren-Chriss + square-root market impact laws

ExecutionAnalytics / OptimalExecution	Almgren-Chriss execution schedule, arrival-price and VWAP slippage
CalibrationPipeline	Nelder-Mead minimization of spread decomposition residuals
BacktestEngine	Historical tape replay against research models
SignalGenerator	Multi-factor composite signal (momentum, spread, VPIN, imbalance)
PaperTrader	Signal-driven IOC order submission with position and P&L tracking
ResearchDashboard / ResearchSerializer	Result visualization and persistence

BCS Latency-Arms-Race Study (bcs_research/)

A separate Python research layer that drives the verified C++ engine through a pybind11 bridge (bcs_engine) to test the Budish–Cramton–Shim theory of the HFT arms race experimentally. The engine is the same one specified in TLA+, but that is **not** a guarantee that observed dynamics are free of matcher error: TLA+ checks the *model*, and the C++ is not the model. Without a refinement mapping or trace validation — logging the implementation's state transitions and checking them against the spec — model-checking says nothing about the binary the experiments actually run. What supports the results instead is ordinary and weaker: the engine's own differential and property tests, and the fact that every qualitative finding survives recalibration. An earlier version of this line claimed the dynamics were "attributable to agent incentives rather than to matcher error" on the strength of the TLA+ work alone; that inference does not hold.

Component	Purpose
agents/	BCSMarketMaker (adverse-selection-aware, latency-disadvantaged), HFTAgent (snipes stale quotes), NoiseTrader (optional lognormal heavy-tailed sizes)
simulation/scheduler.py	LatencyScheduler — per-agent observation and submission latency
metrics/	Welfare decomposition (closed zero-sum identity), liquidity-gap detector, Kyle's lambda
experiments/	Exps 1–4: welfare transfer, latency sweep, HFT-count sweep, batch-auction remedy — plus run_calibrated.py
calibration/	Gap-safe moment extraction from live BTCUSDT-perp order flow and method-of-moments inversion with fill-rate correction
analysis/	Figure generation and the Hawkes endogeneity diagnostic
paper/bcs_paper_draft.md	The write-up (31 pp., 17.4k words — preprint posted to SSRN #6994722; not peer reviewed)

Each experiment is reported twice: at a pre-registered operating point chosen for mechanism clarity, and re-run at environment parameters fitted to 27 days of Binance BTCUSDT-perpetual order flow (37.0M trades, 4.16M book snapshots). Every qualitative finding survives calibration. The calibration also supplies the study's only external check on magnitude: normalised by traded notional, HFT rent is 0.125 bp at the

operating point and 4.42 bp calibrated, spanning the ≈ 0.4 bp latency-arbitrage tax Aquilina, Budish and O'Neill (2022) measure on real exchange message data. This is **consistent with, but not a discriminating test of**, that figure: 0.125–4.42 bp is a 35 $\times$ interval, and a wide interval containing the target is weak evidence — a model predicting 0.05–40 bp would "bracket" it too. It is reported because it is the only external magnitude check available, not because it is a strong one.

Two results are stated as conditional rather than calibrated. The calibrated rent is an upper bound, because a single latency-disadvantaged maker has no competitor to replenish a cleared quote and so absorbs the entire race. And the *incidence* of the transfer turns on a snipe-to-quote size ratio no public feed identifies: at the operating point's ratio the maker bears the loss, while at a ratio of 0.024 the maker's PnL delta is positive for small HFT counts and noise traders bear it instead.

6. Reliability & High Availability

CRC-32 Journaling

- Write-ahead log with atomic CRC-32 integrity on every entry.
- Crash recovery: replay stops at first corrupted/truncated record.
- Atomic checkpoint: snapshots active book state, rewrites journal.
- Deterministic replay via virtual clock (`setExpiryClock(ClockFn)`).
- Auto-rotation: `OB_JOURNAL_MAX_SIZE_MB` env var triggers `engine.checkpoint()` from the main loop when `Journal::needsCheckpoint()` fires.
- Cancel/Modify/CancelReplace records carry `symbolId`. They previously did not, so replay located the target by scanning every book for the order id and taking the first hit — correct only while order ids are globally unique, which nothing enforces (the duplicate check is per-book). Two participants on different symbols with overlapping client-supplied id ranges caused a resting order to silently vanish on recovery. The field already existed and was zero-filled, so `sizeof(JournalEntry)`, the CRC range and the replication wire length are all unchanged; replay prefers the recorded symbol and falls back to the scan for journals written before it.
- The file carries a 24-byte header (magic, format version, record size). A version or record-size mismatch is refused with a message naming both sides, and the journal declines to append rather than extending a file it could not read.
- Checkpointing no longer stalls appends. Building the snapshot — every resting order, plus a durability barrier — used to run with the journal's append lock held. It now runs outside it, with the append counter rechecked before the swap: unchanged means commit with no worker having waited, changed means rebuild under the lock exactly as before.
- `bytesOnDisk()` is tracked at write time rather than asked for. It is consulted on every append (the entry-count threshold in front of it is false 249,999 times in 250,000), and on Linux that was an `fstat(2)` per order inside the global lock.
- A standby following the journal survives a checkpoint. `JournalFollower` tracked its position as an index into a re-read vector, on the premise — stated in its own comment — that "the journal is append-only and entries are immutable once written." True of `appendEntry`, false of `commitRewrite`, which `rename(2)`s a *smaller* file over the same path: a checkpoint writes one record per resting order, far fewer than the history it replaces. The index then pointed past the end, the cursor froze permanently, and `appliedCount()` went on reporting a plausible number while the standby followed nothing. Checkpoints fire automatically at 250k entries, so nobody had to call `checkpoint()` for this to happen. Detection is now by content: the follower remembers record 1 (which file is this) and its last applied record (has the prefix shifted), and `memcmp`s them — sequence numbers cannot work, because the snapshot is built in a truncated temp journal and

renumbered from 1. Detection is only half: a checkpoint contains no deletions, so the book is emptied and rebuilt from the snapshot rather than replayed on top of stale state.

- The same path was dropping ten fields. The follower's `Snapshot` branch called the 6-argument `addOrder`, so a checkpoint turned an iceberg, GTD, pegged or stop-limit order into a plain GTC limit on the standby — `displayQty=0` fails iceberg admission outright, `stopPrice=0` makes a stop fire on the first trade at any price, and GTD loss is invisible until promotion. The test comparator was blind to exactly those fields, so widening the call alone would have gone green while changing nothing; it now compares all sixteen.

Cross-Host Log Replication (wired end-to-end)

- `ReplicationCoordinator` — TCP log-shipping from primary to backup, instantiated in `src/main.cpp` driven by `OB_NODE_ROLE` / `OB_PRIMARY_HOST` / `OB_JOURNAL_PATH` env vars.
- `Journal::onCommit` hook ships each fsync-durable batch to the backup; backup's `applyReplicatedEntry` writes to its own journal so the replica is durable across its own restarts.
- `HeartbeatMonitor` — configurable failure detection timeout.
- `LeaderLease` + `LeaseGrant` propagation — primary broadcasts lease state every heartbeat tick; backup's `tryAcquire` is fenced on local lease expiry, not just heartbeat miss. Prevents split brain under partial network failure (verified by 19-scenario chaos suite: partition, asymmetric partition, 30% packet loss, +30s clock skew — 0 split brain in all).
- Transport auto-reconnect — `receiveLoop` re-runs `connectTo()` against saved host/port after socket loss; ~3 ms recovery in chaos tests. Reconnect uses exponential backoff: 500ms → 30s cap, resets to base on successful connect.
- Snapshot catchup on join — primary streams every resting order via `MatchingEngine::streamSnapshot` when a backup connects; idempotent on receiver.
- Automatic backup promotion on combined heartbeat-miss + lease-expiry; `setReplayModeAllBooks(false)` flips backup into a live primary.

7. Observability & Operations

Observability Features

- **Structured Logging:** Pluggable `StructuredSink` API (Null, JSON Stderr, Capturing). 7 typed `obSink().log()` events wired in `OrderBook.cpp` (accepted/cancelled/rejected/fill/breaker/state); backends hot-swappable via typed helpers in `StructuredLog.h`.
- **Prometheus Metrics:** `MetricsRegistry` with `/prometheus` text-exposition endpoint. Tracks order flow, queue depth, latencies, and journal health.
- **Webhook Alerting:** `AlertDispatcher` with targets for Slack, PagerDuty, or HTTP webhooks.

Admin HTTP Server (Port 8080)

Endpoint	Purpose
GET /health	K8s liveness probe
GET /readyz	K8s readiness probe — HTTP 503 until warmup, then 200; auth-exempt; separate from liveness /health
GET /metrics	Internal counters (JSON)
GET /prometheus	Prometheus text exposition

GET /book?symbolId=0	L2 order book snapshot
GET /otr? participantId=1	Order-to-trade ratio
GET /journal/head	Last committed journal sequence — the chaos suite's no-committed-loss comparison

`AdminServer::start()` reports whether it actually bound. It used to return `void` and give up on a failed bind with a line on stderr, so the engine went on to announce "Ready for traffic" with no admin port listening — the k8s liveness probe then hit nothing and the pod looked dead for an unrelated reason. A failed bind is now fatal at startup, and `port()` reports the bound port so callers can pass 0 and let the OS choose one.

8. Performance Benchmarks

The engine has four latency benchmarks, and they are meant to disagree. Each one holds a different variable fixed, so any single number quoted on its own misrepresents the engine. What each one is for:

Benchmark	Question it answers	Load model	Workload
HonestBenchmark	How fast can the matching path go under conditions chosen to favour it?	Closed-loop	50K orders, seed=42, 100% fill , no cancels
RealisticFlowBenchmark	What does the engine cost on venue-shaped flow?	Closed-loop	500K events, 44% cancel / 46% new / 8% IOC / 2% modify
CoordinatedOmissionBenchmark	What does a client see when arrivals do not wait for us?	Open-loop , paced	RealisticWorkload New+Cancel stream at a fixed offered rate
ColdCacheBenchmark	What does the first touch after an idle gap cost?	Closed-loop	Same RealisticWorkload stream, working set evicted before each timed op

The last two share `benchmarks/RealisticWorkload.h` — a New+Cancel-only stream (15% marketable submissions, 65% of resting orders scheduled for cancel, power-law sizes, 20 participants). That is **not** the same workload as `RealisticFlowBenchmark`, which has its own inline generator with IOC and modify events and 8 participants. The two are not directly comparable to each other, and neither is directly comparable to `HonestBenchmark`.

`HonestBenchmark` is a **floor, not an expected operating number**. It is closed-loop (the harness issues the next order only after the previous one returns) and every submission fills against a dense resting book — a 100% fill rate is the least cancel-like order flow that exists. It is a legitimate, reproducible controlled baseline and it is the flow the three-path decomposition and all optimization history are measured on. It is not what a venue day looks like. The other three exist because it is not.

P50 is the stable per-operation figure; throughput is wall-clock and load-sensitive. (Per-order/per-fill structured logging and event dispatch are sink-/listener-gated, so the hot path stays allocation- and vtable-

free when nothing is attached.)

Which numbers are authoritative

x86 (AWS c6in.metal) is authoritative. Apple Silicon is indicative only. The dev box is an Apple M3 Pro, and its clock resolution is too coarse to resolve this engine's hot path — see the measurement floor below. Every ARM figure in this section is labelled as such and none of them is a headline number or an SLA.

Measurement floor on the dev box. `bench::nowNs()` is `std::chrono::steady_clock`, which on this M3 Pro ticks at **41 ns (smallest non-zero delta; 42 ns median)**. Measured directly: of 2,000,000 back-to-back `nowNs()` calls, **41.7% returned the same value as their predecessor**. Two consequences, both of which apply to every ARM number below:

- 1. **Every ARM latency is quantized to a ~41.67 ns grid.** A reported 208 ns is 5 ticks and carries ±1 tick (±20%); a reported 42 ns is *one* tick and is not a measurement at all, only a statement that the operation finished inside one clock period.
- 2. **Sub-tick operations are counted — FIXED; ARM tables below predate it.** `recordInterval()` used to read `if (end > start) record(...)`, so an operation completing inside one tick left **no sample at all**. That biases every percentile upward, because the discarded samples are exactly the fastest ones: the left tail vanishes and the survivors are renumbered. Measured at 22-32% of samples on the cancel path.

Both recorders now record a sub-tick span as 0 and report the count. The value is not the point — the true duration is somewhere in `[0, tick)` and the clock cannot say where — but sub-tick samples hold the lowest ranks whatever value they carry, so counting them is what makes every percentile *above* the band correctly ranked. A percentile falling *inside* the band now prints `<tick` (below clock resolution) instead of a number. `end < start` remains unrecorded and is counted separately: on a monotonic clock that is a broken measurement, not a fast one.

The ARM tables in this section were measured before the fix, so their sample counts still exclude the sub-tick ops and their percentiles still carry the upward bias described there. They are left as measured rather than retro-adjusted; re-running them is part of the x86 work below.

An x86 TSC tick on a 2.9 GHz part is ~0.34 ns — roughly 120× finer than this box's 41.67 ns — so neither problem arises there. This is why the x86 box is the source of truth.

Validated x86 — AWS c6in.metal (authoritative, standard Release build)

*These x86 figures were recorded on AWS c6in.metal at commit `d2e688c` and were **not re-run in this cycle** — the dev box is Apple Silicon. They are carried here as previously recorded measurements, with that provenance stated.*

Dual-socket Intel Xeon Platinum 8375C @ 2.90 GHz, hyperthreading disabled (`nosmt`), Ubuntu 26.04, Clang C++20 `-O3 -march=native`, `numactl --cpunodebind=0 --membind=0`. 5 stable runs, post-optimization commit `d2e688c`.

Path	What's Included	P50	P90	P99	Throughput
Core matching	OrderBook + STP + WashTrade + LULD	261 ns	620 ns	1,010 ns	2.80M ops/s

Engine wrapper	+ sequence alloc, rate limiter	269 ns	620 ns	1,001 ns	2.74M ops/s
Full-stack journal	+ GroupCommit (batch=64, async io_uring ack on EBS)	615 ns	1,048 ns	3,568 ns	1.28M ops/s

THE 1.28M FIGURE IS NOT A DURABLE-ACKNOWLEDGEMENT NUMBER, AND A VENUE SHOULD NOT PLAN AGAINST IT. Two things separate it from what a venue that cannot lose an order would see.

First, acknowledgement. `enableDurableClientAcks` is **opt-in and refuses in async mode** (see §11), so on the path this row measures an order is acknowledged when it is matched, not when its journal entry is durable. The row is throughput with journalling switched on, not throughput of durable acks.

Second, storage. 1.28M ops/s is EBS with the `io_uring` async ack. The same journal measured on x86 CI caps at **roughly 165–190k appends/s regardless of worker count**, because at the shipped `batch=64` the binding constraint is the `fdatasync` inside the critical section (§11). Those two numbers are not in conflict — they are different disks — but quoting only the larger one without the constraint is how a reader ends up believing a durable venue runs at 1.28M.

Plan against the fsync-bound figure for your storage, then decide whether batch size and outstanding-chain depth move it. Core matching at 2.80M ops/s is the matching path alone and says nothing about durability.

PGO: Clang IR-based profile-guided optimization (profiled on the seed=42 HonestBenchmark workload) takes Path A core matching to **P50 237 ns / P99 910 ns / 3.10M ops/s** — the headline figure. The table above is the standard (non-PGO) Release build.

perf counters — WITHDRAWN pending a correctly-scoped rerun. This line previously read " `perf` (Path A, 50K orders seed=42): IPC 1.34 · 17.8 branch-misses/order · 152 L1-dcache-misses/order · 12,638 instructions/order", and that attribution was wrong.

`scripts/aws_benchmark.sh` wraps `perf stat` around the **whole HonestBenchmark process** and divides by the order count — its own echo line says "counters are whole-run totals". The profiled run passes only `--orders 50000 --seed 42`, with no path filter and without `--no-journal`, so a single process generates 50,000 orders and then runs warmup plus measured passes of Path A, Path B **and Path C, including Path C's `fdatasync`**. Dividing that total by 50,000 and labelling it "Path A" charges the journal path's syscalls and cache traffic to core matching.

The arithmetic gives it away: 12,638 instructions at IPC 1.34 is ~9,430 cycles, ~3.25 µs on a 2.90 GHz part — more than twelve times the 261 ns P50 reported directly above. 152 L1d misses alone exceed that P50 even if every one hit L2.

The counters are not restated here until they are gathered around the measured region alone. `HonestBenchmark --only a` now exists for that purpose; the rerun needs x86 Linux (the dev box is Apple Silicon and has no `perf`).

Apple Silicon (M3 Pro dev machine — indicative, not an SLA)

All figures in this subsection are from a from-scratch Release build (`-O3 -march=native` , `-DBUILD_BENCHMARKS=ON`) on macOS 26.6 / Apple M3 Pro, run for this document. Read them subject to the 41 ns measurement floor above.

HonestBenchmark — closed-loop, 100% fill, seed=42, 50K orders.

Path	P50	P90	P99	P99.9	Max	Throughput
A Core matching	250 ns	542 ns	1,166 ns	1,709 ns	91,917 ns	2.76M ops/s
B Engine wrapper	250 ns	500 ns	791 ns	1,208 ns	43,292 ns	2.95M ops/s
C Full-stack journal	1,750 ns	4,096 ns	1,761,280 ns	3,899,392 ns	10,773,458 ns	24,603 ops/s

The run reports a **100.0% fill rate on all three paths (50,000 of 50,000)** — this is the 100%-fill workload named above, confirmed by the benchmark's own output rather than assumed. Path A and Path B P50 are *identical* at 250 ns (6 clock ticks) because the engine-wrapper overhead is smaller than one tick and this box cannot resolve it; on x86 it is 8 ns (269 – 261 in the table above). Path C is a macOS/APFS `fdasync` artifact and is **not structural** — the same path is 615 ns P50 on Linux x86 with the `async io_uring` ack.

A previous revision of this table reported ~125 ns for Paths A and B. That figure did not reproduce here and is not retained.

RealisticFlowBenchmark — closed-loop, cancel-heavy venue-shaped flow, 500K events, seed=42.

Realized mix over 495,000 timed ops: **43.9% cancel (217,256) / 46.2% new (228,697) / 7.9% IOC (39,328, of which 41.3% filled) / 2.0% modify (9,719)**. Mean resting depth **2,418 orders**, with **0 empty-book no-op cancels** — the book sustains for the whole run, so the cancel path is hitting live orders rather than missing an empty book.

Path	Samples / timed ops	P50	P90	P99	P99.9	Max
Cancel	159,827 / 217,256	42 ns — <i>at the floor, see below</i>	375 ns	667 ns	792 ns	18,208 ns
New	228,693 / 228,697	208 ns	583 ns	708 ns	1,416 ns	8,583 ns
IOC	38,389 / 39,328	208 ns	375 ns	791 ns	1,792 ns	4,792 ns
Modify	9,719 / 9,719	292 ns	667 ns	1,125 ns	1,750 ns	2,625 ns
Combined	436,628 / 495,000	208 ns	458 ns	709 ns	1,250 ns	18,208 ns

The cancel P50 of 42 ns is not a measurement. It is exactly one clock tick, and it means only that the median cancel completes in under 42 ns — the true value is somewhere below the floor and this box cannot say where. **26.4% of cancels (57,429 of 217,256) completed inside a single tick and were dropped from the histogram entirely** in the run tabulated here, so those cancel percentiles are computed over the slower 73.6% and every one of them is biased upward. The same effect costs the combined row 58,372 of 495,000 samples (11.8%), so the combined P50 of 208 ns shown above is an upper bound rather than a centre. **That drop is now fixed** (see the measurement-floor note earlier in this section) — these figures are retained as measured at the time rather than retro-adjusted, and a re-run is listed below. For scale, a like-for-like re-run after the fix recovered 19,426 previously-invisible cancel samples at 200K events and moved

the cancel mean from 174 ns to 135 ns. An x86 TSC resolves the underlying resolution problem; this box cannot. Cancel is the fastest path in the engine and is therefore the one the dev box is least able to measure.

The `Max` column is worth reading alongside the percentiles: an 18 μ s cancel maximum against a 792 ns P99.9 is macOS scheduler preemption, not engine work. There is no `isolcpus` and no core pinning here.

Coordinated omission — `CoordinatedOmissionBenchmark` (ARM, indicative)

What coordinated omission is. A closed-loop benchmark issues its next request only after the previous one has returned. So when the system stalls, the load generator stalls with it and simply issues the next request late — the stall is never charged to any sample. The requests that should have recorded the worst latency are precisely the ones that were never issued. The result is a tail that looks clean because the bad measurements are missing, not because they did not happen. Both the `HonestBenchmark` and `RealisticFlowBenchmark` tables above are closed-loop and have this defect.

`CoordinatedOmissionBenchmark` fixes it by running **open-loop**: it pins an intended start time for every operation at a fixed offered rate (`intended[i] = t0 + i / rate`), then reports two latencies for each one.

- **CO-naive** = `completion - actual_start` — what a closed-loop harness reports. Hides the backlog.
- **CO-corrected** = `completion - intended_start` — charges each operation from the moment it *should* have started, including time spent waiting behind a queue that was already late.

CO-corrected is the honest number, because it is the only one that describes what a client experiences. A client sends at its own cadence; it does not politely wait for the engine to finish the previous order before sending the next. If the engine falls behind, the client's order sits in a queue, and that queueing delay is real latency that the naive number discards.

Rate sweep on this box (200K new orders $\rightarrow$ 305,758 measured ops, seed=7):

Offered rate	Backlogged	Naive P99	Corrected P99	Naive P99.9	Corrected P99.9
100K/s	0.0% (131)	1,166 ns	1,250 ns	4,192 ns	5,280 ns
500K/s	0.2% (616)	834 ns	1,000 ns	2,334 ns	5,536 ns
1M/s	0.4% (1,261)	750 ns	1,000 ns	1,958 ns	6,016 ns
2M/s	6.6% – 54%	—	<i>unstable, see below</i>	—	—
3M/s	5.5% – 99.9%	—	<i>unstable, see below</i>	—	—
4M/s	42.7%	583 ns	27,136 ns	1,458 ns	56,576 ns
6M/s	100.0%	500 ns	16,777,216 ns	1,333 ns	17,039,360 ns

This is the expected shape and it is worth stating plainly what it shows.

Below capacity (100K–1M/s) the two tables nearly coincide at P50 and P99, and the divergence is confined to P99.9 and beyond. Even here the corrected P99.9 exceeds the naive P99.9 — by **1.26x at**

100K/s, 2.37× at 500K/s and 3.07× at 1M/s — so coordinated omission is already understating the far tail on a box that is not close to saturated, and it understates it more as load rises. This regime is reproducible: three repeat runs at 1M/s gave 0.2%/0.5%/0.6% backlog, naive P99 666/792/834 ns, corrected P99 750/1,208/1,208 ns.

At and past capacity the corrected tail explodes while the naive tail does not move. At 6M/s the naive P99 *improves* to 500 ns while the corrected P99 is 16.8 ms — a factor of 33,000. That is the whole point: a closed-loop harness on a fully saturated engine reports its best-looking numbers, because it has stopped measuring anything except service time on an engine that is drowning. 100% of samples were backlogged at that rate, and the corrected P50 of 8.5 ms is simply the queue growing without bound through the run.

The 2M–3M/s knee is not reproducible on this box and is therefore not reported as a number. Three repeat runs at 2M/s gave 42.0% / 48.5% / 54.2% backlog with corrected P99 spanning 2.6 ms to 10.4 ms; three at 3M/s gave 83.4% / 90.1% / 99.9%. An initial run at each rate gave 6.6% and 5.5%. Once the offered rate is near capacity, the outcome on a non-isolated macOS box depends on what else the scheduler is doing, and run-to-run variance exceeds the effect. **Locating the actual knee needs the x86 box with `isolcpus` and core pinning — it is unmeasured.** What this box does establish is that the knee lies somewhere between 1M/s (stable, 0.4% backlog) and 4M/s (42.7% backlog).

Cold cache — `CoLdCacheBenchmark` (ARM, indicative)

Steady-state benchmarks keep the book's hot structures resident in L1/L2 and so report the warm path. The first order after an idle gap, a context switch, or an eviction pays to pull those lines back. This benchmark evicts the working set through a 64 MB buffer before each timed operation (the eviction itself is not timed). 20K orders → 30,965 ops, seed=7:

	Samples	P50	P90	P99	P99.9	Max	Mean
Warm	28,258	250 ns	500 ns	958 ns	4,832 ns	36,542 ns	302 ns
Cold	28,808	2,625 ns	7,648 ns	12,416 ns	34,560 ns	3,531,875 ns	3,855 ns
Penalty		+2,375 ns	+7,148 ns	+11,458 ns	+29,728 ns		

The cold path costs roughly 10× the warm path at P50 on this box. Both numbers are well clear of the 41 ns floor, so unlike the cancel row this comparison is resolvable here even if the absolute magnitudes are not authoritative. A production tail SLA has to budget for the cold number: the warm 250 ns applies only to an engine that has been continuously busy, and the first order after a quiet period is the one that matters most on a market open or after a halt.

Sustained load — `SustainedLoadTest` (throughput only; latency columns are defective)

A 30-second run completed and is reported here for **throughput and memory only**:

- 30,172,625 orders processed in 30.2 s → **999,987 orders/sec** sustained, 0 rejected
- Resident memory 843,520 KB → 1,186,016 KB over the run (+40.6%)
- Process exit code **1** — its own regression gate fired

Its per-window latency columns are not usable and are not reproduced here. Two defects, both in the harness rather than the engine:

1. **The "per-window" latency is actually cumulative since engine start.** The test calls `engine.getAggregateE2ELatency()` once per window, and that function merges the per-thread `LatencyTracker` s, which are never reset (`src/MatchingEngine.cpp:2361`). Each window therefore reports the distribution since the process began, not the distribution during that window. The signature is unmistakable in the output: the reported P50 decays monotonically from ~780 ms in window 4 to ~48 μ s in window 29, which is a cumulative histogram being diluted by later good samples, not a system getting faster. The reported P99 values sit at exact powers of two (1,073,741,824 ns = 2^{30}), which are log-linear bucket edges in the second range.
2. **The offered rate is double the requested rate.** `interOrderNs` is computed once from the global `--rate` , but `producerCount = max(1, numThreads/2)` producer threads each pace themselves at that full rate. With the default `--threads 4` that is 2 producers $\times$ 500K/s = 1M/s offered, which is exactly the 999,987/s observed. The test does not measure the rate it prints.

Consequently its exit-code-1 "regression" verdict is also not meaningful — the threshold is compared against the cumulative aggregate, so once any early window is bad the gate can never clear. **Sustained per-window latency and degradation over time are unmeasured.** Fixing the harness is tracked in §11.

Not yet measured (needs x86)

Gap	Why it is not here
True cancel-path P50	Below the 41 ns ARM clock tick; needs x86 TSC
Engine-wrapper overhead on ARM	Smaller than one clock tick; x86 measures 8 ns
Coordinated-omission knee (saturation rate)	2M–3M/s region is not reproducible without <code>isolcpus/core pinning</code>
CO-corrected tail on x86 at any rate	The CO benchmark has never been run on the x86 box
<code>RealisticFlowBenchmark</code> on x86	Never run there; all realistic-flow numbers above are ARM
<code>ColdCacheBenchmark</code> on x86	Never run there
instructions/order, IPC, L1d misses	No perf or HW counters on Apple Silicon (see the withdrawal note above)
Sustained per-window latency	Harness defective — see above

The ARM-vs-x86 gap on core matching is microarchitectural (wider out-of-order window + stronger branch prediction on pointer-chasing code), **confirmed not** caused by build flags, field ordering, branch hints, or branchless selection.

Where the 261 ns goes (and what doesn't move it)

The four earlier micro-fixes (listener-dispatch guard, `shared_mutex` $\rightarrow$ plain `mutex` , OCO scratch-buffer reuse, rehash guard) produced **no measurable x86 latency change**. The conclusion drawn from that — that the P50 is structurally bound rather than instruction-bound — was supported by the per-order counters above, which are now withdrawn as mis-scoped. The *observation* stands (four changes, 0 ns); the *mechanism* is unproven until the counters are regathered. This cycle's branchless price-cross *did* move it, shaving 10 ns P50 / 62 ns P99 to reach 261 ns (see Optimization History in BENCHMARKS.md); next-order prefetch and the price-level arena allocator were both implemented and reverted as net-negative on this 100%-fill flow.

THIS ENTIRE TABLE IS WITHDRAWN, NOT JUST ITS FIRST ROW. An earlier edit struck the 152 L1-dcache misses/order citation and left 17.8 branch-misses/order standing one row below — the same mis-scoped counter, from the same perf run. And the ns column is not an independent measurement at all: that split was apportioned from those counters, so withdrawing them withdraws the attribution built on top of them. Striking one cell and leaving its siblings is a worse state than striking none, because it reads as though the rest survived review.

The table is kept, struck, because the levers in the right-hand column were measured independently of the counters and are still good: the branchless price-cross A/B is a real shipped -10 ns P50 / -62 ns P99, and the arena and prefetch verdicts were re-measured properly (see BENCHMARKS.md). What is gone is any claim about where the 261 ns goes.

Closing this needs the scoped x86 re-run: HonestBenchmark --only a under perf stat , one path per run — the mechanism aws_benchmark.sh now uses.

Cost	ns	Driver	Lever (independently measured)
Pointer chasing (intrusive list)	80 100	152 L1-dcache misses/order	arena allocator — re-measured and still reverted : 2.5–3.6% worse on cancel-heavy flow (p≈0.002), +16% mean / -13% throughput on 100%-fill (p<1e-5)
Branch mispredicts	60 80	17.8/order, data-dependent	branchless price-cross — shipped, -10 ns P50 / -62 ns P99 (a real A/B, not derived from the withdrawn counters)
Irreducible work	50 60	price/qty math, STP, compliance	—
Spectre mitigation (eBRS)	30 40	kernel-enforced on this instance	not disableable on this instance type

Confirmed **0 ns delta** on this workload: -02 vs -03 , Order field reordering, [[likely]] / [[unlikely]] hints, branchless isBuy book selection. Shipped this cycle: branchless price-cross (-10 ns P50 / -62 ns P99) and io_uring async journal ack (Path C P99 5,312→3,568 ns); next-order prefetch and the price-level arena allocator were both reverted as net-negative. Clang IR-based PGO then took Path A to 237 ns P50 (the headline figure). The journal P99 (~3.6 μs) is the fdatsync /EBS flush; NVMe/RAM-backed storage would be materially lower.

Binary Codec Microbenchmark

Codec	Message	ns/op	M ops/s
OUCH 4.2	EnterOrder encode (49B)	112.0	8.9
ITCH 5.0	AddOrder encode (36B)	34.1	29.4
SBE	NewOrderV1 encode (32B)	1.0	1015
SBE	NewOrderV1 decode (32B)	0.5	2128

Read these as throughput ceilings, not per-message costs. 0.5 ns is ~1.45 cycles at 2.9 GHz, which is not the latency of decoding a message — it is what a tight loop achieves over an L1-resident buffer when the

CPU can overlap successive iterations. The work is real (`benchmarks/BinaryCodecBenchmark.cpp` applies `doNotOptimize / clobber` inside every loop, so nothing is eliminated), but a decode of a message that just arrived from the network — cold line, unpredictable branch — will not hit this figure.

The OUCH row is ~325 cycles for 49 bytes and looks anomalous next to the others. It is not: OUCH carries **ASCII-decimal** fields on the wire, so its encode cost includes integer-to-string formatting, while SBE uses native-endian binary and does none. The benchmark prints this in its own notes. The table compares three protocols doing genuinely different work, not one implementation outperforming another.

9. Verification & Testing

Test Suite — 102 Executables, 524 CTest Targets

The testing infrastructure includes Unit, Functional, Integration, Chaos, Property, Shadow, and Benchmark testing categories across 102 test executables and 524 CTest targets.

The count is not the claim, and it should not be read as one. There are no line- or branch-coverage figures here and no mutation testing, so the number measures how many test binaries exist, not how much behaviour they pin. This project has repeated evidence that the two diverge: every journal test was single-symbol, which is precisely how the cancel-routing defect survived; a follower comparator checked 6 of 16 order fields, so a fix that dropped ten of them would have gone green; the checkpoint soak ran 90 rewrites that all *grew* the file, never once producing the shape a real checkpoint makes; and eight test functions were defined and never called. All four are fixed, and all four were inside that count while it was being quoted. Key mechanisms:

- **Shadow Mode:** Dual-book divergence detection, validating FIFO compliance.
- **Fault Injection:** 10+ injection points (short-writes, pool exhaustion, EAGAIN injection) with zero-cost overhead in production.
- **Coverage-Guided Fuzzing:** libFuzzer harness for protocol parsing and order flow.
- **Sanitizers:** ASan, UBSan, and TSan checks integrated into CI/CD.
- **Multi-symbol journal coverage:** replay equivalence is swept over 20 seeds × 250 ops across three symbols. Every journal test was previously single-symbol — the one shape in which cross-book routing cannot be wrong — which is how the cancel-routing defect above survived.
- **Journal contention measurement:** `JournalContentionBenchmark` reports append throughput against worker count with journaling on and off, and against batch size so the durability barrier can be amortised away and the lock itself becomes visible. Run on x86 via the `JournalContention (H6)` workflow, which records CPU, filesystem and measured `fdatasync` latency first, because every number is relative to those.
- **Local Linux reproduction:** `scripts/verify_linux.sh` runs a CI lane in an `ubuntu:24.04` container (`sanitizers | tsan | release | faultinject`). macOS misses `-Werror`, uses `kqueue` rather than `epoll`, links `libc++` rather than `libstdc++`, and cannot compile the `io_uring` path at all — so a clean local build has never been evidence, and every one of those differences has been found by CI rather than before the push. The tree is mounted read-only and the build lands inside the container, so a Linux build cannot leave objects behind for a later macOS build to link against.
- **Checkpoint soak exercises both directions:** `CheckpointChaosTest` previously ran 90 rewrites that all GREW the file, so the shape a real checkpoint produces — one record per resting order, replacing a far longer history — was never tested. Rewrite targets now alternate, and the test asserts that a shrink actually occurred, because the alternation is otherwise silently removable. That gap is why a consumer tracking its position as a positional index went unnoticed.

Formal Verification (TLA+)

12 TLA+ specifications, model-checked with TLC:

- **MatchingEngine.tla** : 171,187,419 distinct states verified at `MatchingEngine4.cfg` (MaxOrders=4), zero violations — now including the matching/cross layer (`Match` action) with `MatchingConservation` and `FIFOExecution` , alongside `NoNegativeQuantity` , `FIFO_Preservation` , `GTD_Expiry_Correctness` .
- **Replication.tla** : Realistic lease-propagation model (no god-mode `~primaryAlive` guard on `BackupPromote` ; promotion requires both heartbeat-miss AND local-lease-expiry). 1373 → 4192 states verified at `MaxEntries=6 → 10` , zero violations. A bug-injected variant (lease check stripped) reproduces split brain in 188 states, confirming the verification is genuine.
- **MpscQueue.tla** : Lock-free ring buffer linearizability.
- **SnapshotLocked.tla** : Mutex torn-snapshot prevention.

10. File Structure

include/	93 header files – core logic and networking
src/	14 source files – thin compilation units
tests/	103 test files, 524 CTest targets
benchmarks/	11 benchmark binaries
fuzz/	3 libFuzzer harnesses + standalone driver
spec/	TLA+ formal specifications (12 specs)
tools/	CLI tools and data utilities
config/	Example configuration files
docs/	Runbooks, CapacityPlanning, ProductionReadiness

11. Remaining Work

The system is architecturally complete. The remaining items are AWS validation steps for capabilities already implemented and locally verified (nothing is hardware-blocked — the kernel-bypass path runs on commodity AWS ENA hardware):

- **x86 Bare Metal Benchmarks:** ✅ done — validated on AWS c6in.metal (dual Xeon 8375C, `nosmt` , NUMA-pinned); see §8. Core matching 261 ns P50, confirming the structural-bottleneck analysis (pointer-chasing L1 misses + data-dependent branch mispredicts + Spectre eIBRS). The prior four micro-optimizations moved x86 latency 0 ns; this cycle's branchless price-cross shaved 10 ns while next-order prefetch and the arena allocator were both reverted as net-negative — confirming the P50 is structurally bound.
- **Journal Async I/O (io_uring):** ✅ done — validated on x86 Linux (AWS c6in.metal). The async ack (onCommit fires on the completion-reaper thread, not on submit) cut Path C P99 **32.8%** (5,312 → 3,568 ns) at the cost of +167 ns P50. `io_uring` is a generic Linux async-I/O interface and needs no special NIC — only `liburing` on a modern kernel; the seam stays behind `#ifdef __linux__` with an `fdasync / F_FULLFSYNC` fallback elsewhere.
- **Kernel Bypass Networking (DPDK / F-Stack):** ⌚ **written but never executed** — no run, no measurement, no demonstrated benefit. `DpdkGateway` integrates F-Stack (DPDK userspace TCP over the AWS ENA PMD) on a secondary ENI behind `#ifdef OB_HAVE_DPDK` , feeding `OuchSession::feed()` unchanged (see §3). The integration is written and the tree builds with the path `#ifdef 'd` out — which verifies only that the code is *absent* cleanly, not that it works; `scripts/dpdk_aws.sh` provisions the receiver (hugepages, DPDK + F-Stack install, vfio-pci bind, F-Stack config). It runs on **commodity AWS ENA** hardware — no Solarflare/Onload — needing only

a secondary ENI, hugepages, and the DPDK/F-Stack toolchain; with the kernel-TCP wire-to-wire baseline now measured (see below), the two-instance c6in.metal run — **Session 2**, the DPDK-vs-kernel-TCP comparison — is the next step.

- **Replication.tla Verification:**  done — see Verification section above. The original "not yet verified" footnote is obsolete.
- **BCS Study — Market-Maker Enrichment:**  open. The environment calibration (§5) is complete and the full experimental grid has been re-run at fitted parameters, but the model still has a single market maker. That is why the calibrated rent is reported as an upper bound rather than an estimate, and adding competing makers with heterogeneous latencies is the single change that would most improve the study's external validity. A second open item is the Hawkes diagnostic's real-data leg, which needs a depth-depletion gap definition for books that never empty.
- **Wire-to-Wire Latency Validation:**  partially complete — baseline measured, DPDK comparison pending. The `WireLatencySender / WireLatencyReceiver` harness (`SO_TIMESTAMPING` hardware timestamps with `CLOCK_MONOTONIC` software fallback, round-trip/2 one-way estimate — see §3) is validated on AWS: a two-instance run (two `c6in.metal`, same AZ, kernel TCP, 10,000 OUCH orders, software timestamps on `CLOCK_MONOTONIC`) measured **P50 RTT 48.2 μ s / P99 RTT 55.3 μ s**, i.e. a **P50 one-way estimate of 24.1 μ s**. The remaining step is the DPDK-vs-kernel-TCP comparison (**Session 2**) via `scripts/wire_latency_aws.sh`.

Open engineering items

Distinct from the AWS validation steps above: these came out of an adversarial audit of the codebase and are tracked as code work.

- **Durable client acknowledgements on the async path** — open, and documented rather than built. On the sync path an order is matched before it is acknowledged, and with `enableDurableClientAcks` the resulting fills are withheld until the journal entry behind them is fsync-durable. On the **async** path `submitOrder` returns at enqueue: the order has been QUEUED, nothing has matched, nothing is journalled. `enableDurableClientAcks` refuses in async mode rather than gating the fills while leaving that ack as undurable as it was — advertising a guarantee that is not there would be worse than declining to offer it. This is stated on `SubmitResult` itself so a caller meets it where they use it.

It is not, as previously recorded here, a protocol decision. Closing it needs one `DurabilityGate` per concurrently-processing thread — the gate's capture model holds a single in-flight order, so N workers overwrite each other's groups and a lock serialises that rather than preventing it — plus a drain that runs when the engine is idle, because durability becomes true at moments when no order is being processed (an explicit flush, a later order filling the batch, a checkpoint). That is a few hundred lines touching the event-dispatch path on every book, and it is worth doing only if real order flow runs through the async path; the sync path is already correct for a venue that needs the guarantee.

- **Checkpoint as a log record** — open, and scoped to *don't build*. The checkpoint replaces the journal rather than appending to it, so an entry committed between the snapshot gather and the swap is discarded. The append-counter check added above narrows this to a detected, logged fallback rather than a silent loss, but closing it properly means writing the snapshot as a record *in* the log and replaying forward from it, the way a WAL checkpoint works. That is a format change, not a locking change.
- **Journal record framing** — resolved. The file previously had no magic number, length prefix or version field: framing was implicit in `sizeof(JournalEntry)`, so a layout change sliced an existing file on the wrong boundaries, failed every CRC, returned an empty replay, and — because

the file is opened "ab+" — appended to it anyway. Silent, total, and on the upgrade path. A 24-byte header now carries a magic, a format version and the record size, and a mismatch is refused by name rather than inferred from every record failing at once. Journals written before the header are still read as bare records: their first byte is an `entryType` (1–5) and the magic begins with '0', so the two are distinguished with certainty rather than by probability. The header is written lazily, immediately before the first record, so that constructing a `Journal` purely to read — as `JournalFollower`, `ResearchHarness` and the CLI tools do — still does not modify the file.

- **Journal append serialisation** — measured, largely resolved. All appends serialise on one mutex, but at the shipped batch size the binding constraint is the `fdasync` executed inside the critical section, which caps throughput at roughly 165–190k appends/s on x86 CI regardless of worker count. The per-append syscall has been removed; the remaining lever is batch size and outstanding-chain depth in `Journal`, not resharding the log.
- **Alerting components not wired in** — half closed. `CapacityMonitor` now runs inside the real engine: `startAsync()` installs the resource callbacks and starts it, `stopAsync()` stops it first so it cannot outlive the queues its callbacks read, and a breach emits `capacity_alert` to the same structured sink that already carries `checkpoint_abandoned`. Previously a breach with no dispatcher attached produced nothing at all but a counter increment. Two probes need no policy and are live — worst-of queue depth across the request rings, and journal-filesystem usage (`capacity - available`, since root-reserved blocks are not usable; the path is resolved once at install so the monitor thread never takes the journal lock). A probe failure reports once rather than at 1 Hz, because returning 0.0 silently reads as "disk empty", which is indistinguishable from healthy.

What remains needs decisions, not effort. **Memory**: the threshold is a fraction of a *limit* and no memory ceiling exists anywhere in config, flags or code — RSS over host RAM is the wrong denominator for a process sharing a box. **Replication lag**: `ReplicationCoordinator` is constructed after `startAsync()`, so there is nothing to read at install time; closing it is a startup-ordering change. **IncidentLogger**: no config knob exists, and inventing a path is worse than leaving it shut. **AlertDispatcher**: now wired. `alert_webhook_url` / `alert_webhook_format` / `alert_min_level` / `alert_routing_key` existed only as commented-out lines nothing read, so the dispatcher was reachable from no production path; `src/main.cpp` now reads them and attaches it to both `CapacityMonitor` and the graduated kill switch. It still has **no TLS client** — and rather than leave that to be discovered mid-incident, an `https://` URL is now refused **at startup** and the engine declines to boot, with the message naming the fix. The engine does not gain a TLS stack for this on purpose: it has no external dependencies beyond `liburing` on Linux, and linking OpenSSL to deliver one webhook would trade that for a supply-chain surface on the alerting path. TLS terminates in a local sidecar (`engine --http--> 127.0.0.1 --https--> Slack`), which also keeps a credential-bearing URL out of the engine's config. The shipped example previously pointed at `https://hooks.slack.com/...`, a URL this code has never been able to deliver.

Two shipped thresholds look wrong and are flagged rather than quietly edited, since they are policy: `checkIntervalMs = 1000` against an 8192-slot ring that fills in roughly 80 ms at 100k msg/s makes the queue-depth alert a saturation detector rather than a burst detector, and `replicationLagMs = 100` is exactly `heartbeat_interval_ms` in the example config.

- **Standby symbol routing** — resolved. `JournalFollower` ignored `symbolId` on every branch, so a multi-symbol leader collapsed into one follower book where order ids from different symbols crossed against each other and the standby printed fills the leader never had. It now takes either an explicit `SymbolId` or a resolver; the symbol argument sits before the book so the old two-

argument call fails to compile rather than silently binding the poll interval as a symbol. A skipped record still advances the cursor, since the counter is a position in the file rather than a count of mutations. Two traps had to be cleared before the test could catch the bug at all — the volatility collar rejected the misrouted order, and self-trade prevention cancelled the crossing one, each making the test pass for the wrong reason.

- **Follower poll cost** — resolved. The follower re-read and re-CRCed the entire journal on every 1 ms poll, which at the 250k-entry checkpoint threshold is tens of MB per millisecond. It now reads record 1, the prefix anchor and the new tail through a **single** `fopen`, so nothing can straddle a `rename(2)` between "which file is this" and "what is in it" — the window that ruled out an `st_ino` check. Measured at 2 records per poll against a 1000-record file, flat in file size. Constructing a `Journal` per poll is gone too; that opened the file `"ab+"` and re-walked it to recover the sequence, so a read-only follower was scanning the log twice per poll and opening it for append.
- **SustainedLoadTest reports cumulative latency as per-window** — open, found while gathering §8. The test calls `engine.getAggregateE2ELatency()` once per window and prints the result as that window's P50/P99/P99.9, but that function merges per-thread `LatencyTracker`s that are never reset (`src/MatchingEngine.cpp:2361`), so every window reports the distribution since process start. The tell is that the printed P50 *decays* monotonically across a run — from ~780 ms to ~48 μ s over 30 windows — which is a cumulative histogram being diluted by later samples, not a system speeding up. Because the regression gate compares that cumulative aggregate against `--p99-threshold`, one bad early window latches the gate to failing for the rest of the run: the test returns 1 regardless of whether degradation is still occurring, which makes it useless as a CI signal in both directions.

A second, independent defect: `interOrderNs` is derived once from the global `--rate`, but each of the `max(1, numThreads/2)` producer threads paces itself at that full rate, so the offered load is `producerCount × rate`. At the defaults (`--threads 4 --rate 500000`) the run offers 1M/s and measures 999,987/s. Closing both means giving the engine a windowed latency snapshot (or resetting the trackers per window) and dividing the rate across producers. Until then the test's throughput and memory figures are usable and its latency columns are not.

Developed for professional quantitative trading systems. C++20 · 102 test executables · 524 CTest targets · 19 multi-container chaos scenarios · 171M distinct TLA+ states verified on the matching-inclusive MatchingEngine.tla · 12 TLA+ specifications